La consultation des données

Table des matières

1. Objectif de l'activité professionnelle 1
2. La structure de données 1
3. La consultation des catégories 2
   3.1. L'interface 2
   3.2. La classe métier Categorie 2
   3.3. Le script index.html 3
   3.4. Le contrôleur categorie/liste/index.php 3
   3.5. Le script index.js 3
4. Les coureurs 4
   4.1. L'interface 4
   4.2. La classe Coureur 4
   4.3. Le script index.php 5
   4.4. Le script index.html 5
   4.5. Le script index.js 5
5. Les clubs 6
   5.1. L'interface 6
   5.2. La classe Club 6
   5.3. Le script index.html 8
   5.4. Le script index.php 8
   5.5. Le script index.js 8
6. Les annonces 8
   6.1. L'interface 8
   6.2. La classe Annonce 9
   6.3. Le script index.html 9
   6.4. Le script index.js 9
7. Les projets 10
8. Les compétences associées à un projet 11
   8.1. L'interface 11
   8.2. Le script index.js 11
   8.3. Le contrôleur ajax/getlescompetences.php 12

Guy Verghote

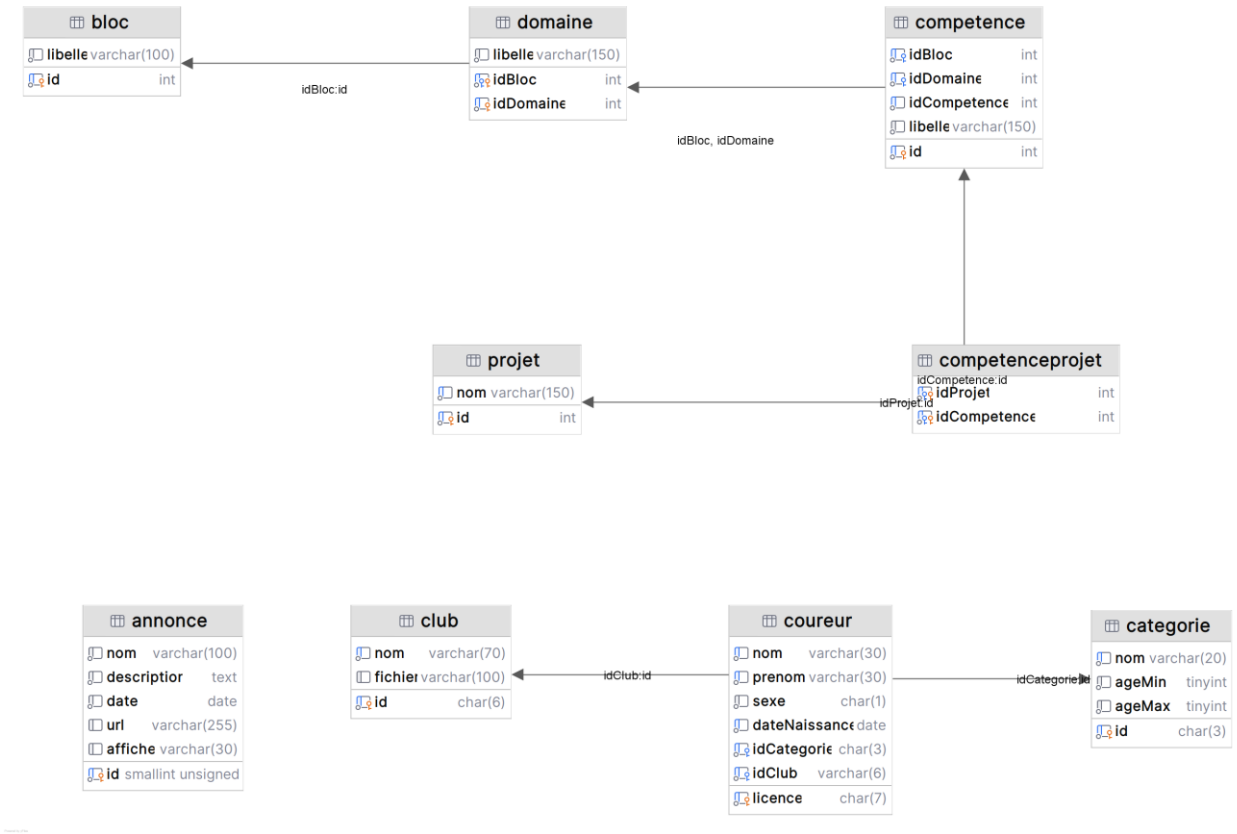
La consultation des données

1. Objectif de l'activité professionnelle

Étudier l'accès en consultation aux données d'une table Étudier les différentes solutions de mise en forme des données côté client (tableau, carte) Être capable de récupérer et d'afficher des données sur d'autres développements

2. La structure de données

Schéma de la base de données consultation à obtenir



1. Mettre en place la structure de données en exécutant les scripts SQL dans l'ordre ci-dessous.

/sql/create database.sql
/sql/user.sql
/sql/coureur/create.sql (puis insert.sql),
/sql/projet/create.sql (puis insert.sql),
/sql/annonce/create.sql (puis insert)

2. Vérifier la conformité de votre structure en lançant en mode CLI les scripts ci-dessous

/maintenance/Contrôle de la structure de données/controler.php
/maintenance/Contrôle des fichiers de données/controler.php
/maintenance/Contrôle des fichiers de données/supprimer_orphelin.php (si nécessaire)

3. Générer sous PhpStorm le diagramme de la base et vérifier sa correspondance avec celui fourni.

02 Développement Web> 01 Présentation > 03 Architecture du mini framework

3. La consultation des catégories

3.1. L'interface



Liste des catégories pour la saison en cours — Télécharger le tableau en PDF

Catégorie	Code	Âge entre	Né(e) entre
Minime	MI	14-15	2012-2013
Cadet	CA	16-17	2010-2011
Junior	JU	18-19	2008-2009
Espoir	ES	20-22	2005-2007
Senior	SE	23-34	1993-2004
Master 0	M0	35-39	1988-1992
Master 1	M1	40-44	1983-1987
Master 2	M2	45-49	1978-1982
Master 3	M3	50-54	1973-1977
Master 4	M4	55-59	1968-1972
Master 5	M5	60-64	1963-1967
Master 6	M6	65-69	1958-1962
Master 7	M7	70-74	1953-1957

3.2. La classe métier Categorie

Elle assure le dialogue entre le contrôleur et la base de données.

Elle contient les méthodes permettant d'interroger la table 'categorie'.

Chaque méthode :

- se connecte à la base,
- prépare et envoie la requête SQL au serveur MySQL
- récupère et retourne la réponse du serveur

Important : **Seules les méthodes des classes métier peuvent contenir des instructions SQL**

Pour éviter les redondances dans l'utilisation des classes et méthode de l'API PDO, la classe Select est utilisée.

02 Développement Web> 03 Élément pratiques > Classe technique > 02 Les classes en liaison avec la base de données > 01 La classe Database

02 Développement Web> 03 Élément pratiques > Classe technique > 02 Les classes en liaison avec la base de données > 01La classe Select

3.3. Le script index.html

Il décrit la partie statique de la page, l'utilisation du composant html2pdf qui génère une version PDF de la page, nécessite d'encapsuler la partie de l'interface à générer au format PDF dans une balise

Il contient principalement la balise <table> avec la définition de l'entête du tableau et la balise tbody, identifiée par l'id 'lesLignes' afin de pouvoir la remplir dynamiquement.

3.4. Le contrôleur categorie/liste/index.php

Le fichier index.php constitue le point d'entrée (Endpoint) de cette fonctionnalité. Il joue le rôle de contrôleur dans l'architecture MVC. Son rôle consiste à préparer les données nécessaires à l'affichage de l'interface, à transmettre ces données au navigateur et à demander ensuite le chargement du template commun view/interface.php.

Le traitement réalisé par un point d'entrée se déroule toujours selon les quatre étapes suivantes :

1. Initialiser l'environnement d'exécution.
2. Récupérer les données à transmettre côté client.
3. Utiliser les méthodes de la classe Page pour construire et générer la page

Il initialise l'application via bootstrap.php, récupère les catégories auprès de la classe métier Categorie, puis transmet ces données à l'objet Page. Il ajoute également la bibliothèque JavaScript html2pdf afin de permettre la génération d'un document PDF à partir de la page HTML, avant d'afficher la page.

Ici nous utilisons le composant html2pdf pour obtenir la conversion de la page HTML en fichier PDF. Ce composant est pratique lorsque la page HTML tient sur une seule page PDF

02 Développement Web> 03 Élément pratiques > Classe technique > 01 Les classes présentes dans tous les projets > 01 La classe Page

4. **Afficher le code source de la page afin de visualiser les données transmises.**

3.5. Le script index.js

Le script index.js génère contenu de la balise <tbody id='lesLignes'>
Il s'agit de parcourir chaque ligne du tableau 'lesCategories' afin de générer une balise tr contenant 4 balises td.
Indication : la feuille de style style.css définit la mise en forme des balises table, tr et td.

Toutes les interfaces devant afficher les données dans une balise <table> vont répéter cette méthode de construction : insertion d'une ligne dans laquelle on insère des colonnes avec une valeur et une mise en forme. L'utilisation de la bibliothèque de fonctions trtd.js permet de simplifier le code en utilisant les méthodes creerTr et creerTd

02 Développement Web> 03 Élément pratiques > Composant client > trtd.js

La génération de la version PDF se déclenche sur le clic du bouton btnGenerer :

4. Les coureurs

Répertoire : consultation/coureur

4.1. L'interface

Ordinateur

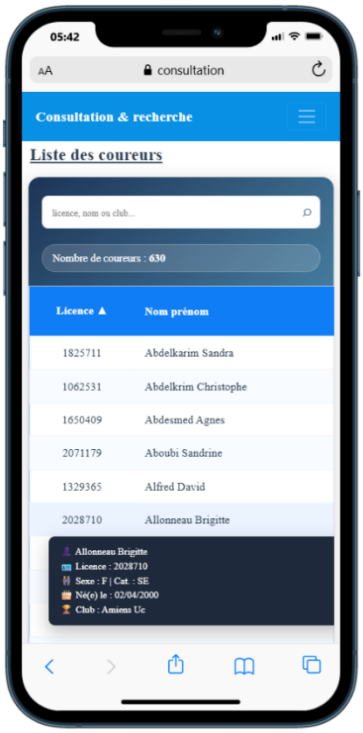
Liste des coureurs

Nombre de coureurs : 630

Licence, nom ou club...

Licence ▲	Nom prénom	Sexe	Né(e) le	Cat.	Club
1825711	Abdelkarim Sandra	F	25/12/1978	M2	Vytajog
1062531	Abdelkrim Christophe	M	30/08/1957	M6	Amicale Du Val De Somme
1650409	Abdesmed Agnes	F	25/09/1996	SE	Amicale Du Val De Somme
2071179	Aboubi Sandrine	F	22/10/1996	SE	Albert Meaulte Aerospa.ac
1329365	Alfred David	M	02/10/1982	M1	Amicale Du Val De Somme
2028710	Allonneau Brigitte	F	02/04/2000	SE	Amiens Uc
1086136	Andre Stephane	M	16/02/1971	M4	Amicale Du Val De Somme
2095355	Andrieux Emeline	F	20/05/1992	SE	Us Camon
1755734	Ansard Ahmed	M	26/09/1992	SE	Amiens Uc
1864512	Avisse Christophe	M	23/06/1977	M2	Sporting Club Abbevillois Athl
2044892	Aymard Corinne	F	22/02/1983	M1	Amicale Du Val De Somme
1291516	Baecher Benjamin	M	29/05/1978	M2	Amicale Du Val De Somme
1851962	Baillet Mohamed	M	26/06/2008	JU	Us Camon

Smartphone



4.2. La classe Coureur

La méthode getAll() permet de récupérer l'ensemble des données.
La mise en forme de la date s'effectue ici au niveau de la requête SQL, c'est de loin la façon la plus simple :

```
SELECT licence, nom, prenom, sexe, date_format(dateNaissance, '%d/%m/%Y') AS dateNaissanceFr, idCategorie, nomClub FROM coureur
```

Cela pourrait se faire en PHP mais il faudrait modifier en conséquence le code, en formatant après fetch :

```
$dateFr = (new DateTime($row['dateNaissance']))->format('d/m/Y');
```

Cela pourrait se faire en Javascript, en formatant côté client :

```
new Date(row.dateNaissance).toLocaleDateString('fr-FR').
```

Parmi les 3, la recommandation reste la solution SQL pour une simple liste en lecture : aucune boucle supplémentaire, pas de problème de fuseau horaire, et on garde le tri sur la vraie colonne dateNaissance en base.

4.3. Le script index.php

Il charge les données et instancie un objet Page pour construire la page affichée.

4.4. Le script index.html

Il contient les styles spécifiques qui permettent de modifier l'aspect de la page en fonction de la taille de l'écran.

Les attributs data-champ et data-type sont utilisés par la fonction activerTri du composant /composant/fonction/tableau.js

02 Développement Web> 03 Élément pratique > Composant client > trtd.js

4.5. Le script index.js

index.js gère l'affichage, le filtrage et le tri d'un tableau de coureurs dans une interface web.

Il utilise les fonctions 'externes' suivantes

creerLigneCoureur (depuis coureur.js) : Fonction qui génère une ligne de tableau (<tr>) pour un coureur, avec ses informations formatées.
activerTri (depuis tableau.js) : Fonction qui active le tri interactif sur les colonnes du tableau.
getData (depuis page.js) : Récupère les données des coureurs injectées par Vue.

Il définit les constantes suivantes :

lesLignes : Conteneur HTML (<tbody> ou similaire) où sont insérées les lignes des coureurs.
nb : Élément HTML affichant le nombre de coureurs filtrés.
search : Champ de recherche pour filtrer les coureurs.
lesCoureurs : Tableau des coureurs récupéré via getData('lesCoureurs').

À chaque saisie dans search, la fonction afficher(lesCoureurs) est appelée.
Elle parcourt la liste des coureurs et filtre ceux dont la licence, le nom ou le club contient la valeur saisie (insensible à la casse).
Les coureurs correspondants sont affichés via creerLigneCoureur, et le compteur nb est mis à jour.

activerTri() est appelée avec un objet de configuration :

idTable: "leTableau" : Identifiant du tableau HTML à trier.
getData: () => lesCoureurs : Fonction qui retourne les données à trier (ici, le tableau lesCoureurs).
afficher: afficher : Fonction de rappel pour rafraîchir l'affichage après un tri.
triInitial : Tri par défaut sur la colonne licence (ordre ascendant).

02 Développement Web> 03 Élément pratique > Composant client > tableau.js

Quand l'utilisateur clique sur une colonne, activerTri trie les données selon cette colonne, puis appelle afficher pour mettre à jour le tableau.
Le script coureur.js contient la fonction creerLigneCoureur(coureur) :
Elle formate les données du coureur (ex. : ucWord pour mettre en majuscule la première lettre des noms).
Elle construit une ligne de tableau (<tr>) avec les cellules (<td>) contenant : Licence, nom, sexe, date de naissance, catégorie, club.
Un attribut data-details pour afficher une infobulle avec les détails du coureur (utile sur mobile).

Un attribut tabindex="0" pour permettre l'interaction au clic/toucher sur smartphone.

En résumé
index.js : Gère l'affichage, le filtrage et le tri du tableau de coureurs.
coureur.js : Fournit une fonction pour créer une ligne de tableau avec les données d'un coureur, formatées et adaptées pour mobile.
Interaction : Le tri et le filtrage sont dynamiques et déclenchés par les actions de l'utilisateur.

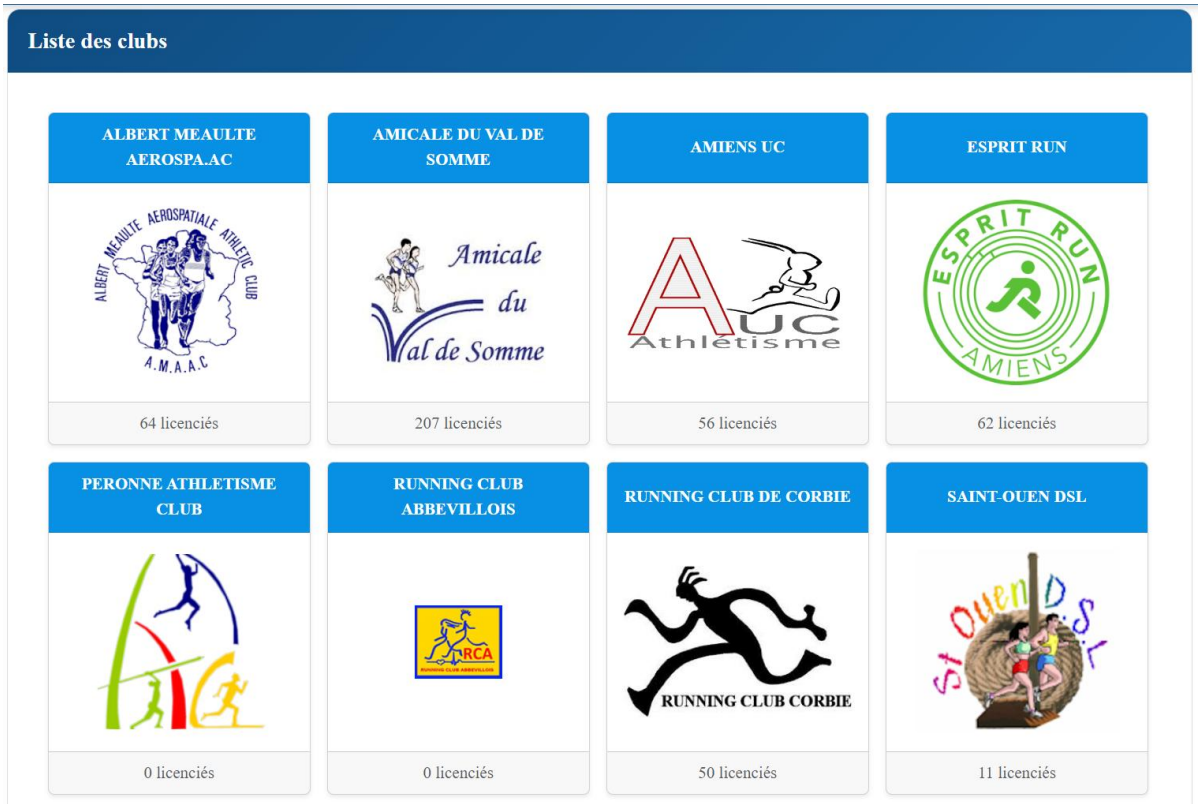
5. Vérifier le fonctionnement de cette interface et son adaptation sur smartphone.

5. Les clubs

Répertoire : consultation/club

Il est parfois préférable d'afficher les données sous forme de carte à l'aide des classes Bootstrap 'card'. C'est le cas lorsque les données sont peu nombreuses ou comportent des images. Nous allons afficher les clubs de cette façon.

5.1. L'interface



Le champ fichier de la table club contient le nom du fichier image associé au club (son logo).
Les logos sont stockés dans le répertoire /data/club

5.2. La classe Club

Nous avons besoin de récupérer dans la base de données les colonnes id, nom, fichier et le nombre de licenciés de chaque club. Tous les clubs doivent apparaitre.

Pour obtenir le nombre de licenciés de chaque club, nous pouvons utiliser une sous requête dans la clause 'select'

```
select id, nom, fichier, (select count(*) from coureur where coureur.idClub = club.id) as nb
from club
order by nom;
```

Cependant l'emploi d'une jointure par requête corrélée, est souvent moins performant qu'une jointure externe associé à la clause Group by

```
select club.id, club.nom, fichier count(*) as nb
from club
  left join coureur on coureur.idClub = club.id
group by club.id, club.nom, fichier
order by club.nom;
```

Il est souhaitable de réaliser un contrôle supplémentaire permettant de s'assurer que la colonne fichier contient une URI (Uniform Resource Identifier ou Adresse d'une ressource physique) valide. En d'autres termes, s'assurer que le logo est bien présent dans le répertoire data/club afin d'éviter des erreurs lors de l'affichage côté client.

Pour cela il faut parcourir le résultat de la requête et lui ajouter une colonne 'present' de type booléen qui indiquera si la ressource existe bien.

La colonne fichier de la table club accepte la valeur null (tous les clubs n'ont pas forcément de logo)

```
// ajout d'une colonne permettant de vérifier l'existence du logo
foreach ($lesLignes as &$ligne) {
  $ligne['present'] = isset($ligne['fichier']) && $ligne['fichier'] !== '' &&
                      is_file(self::DOSSIER_LOGO_CLUB . $ligne['fichier']);      }
```

Sans utiliser le symbole & qui permet de passer la variable par référence :

```
foreach ($lesLignes as $index => $ligne) {
   $lesLignes[$index]['present'] = isset($ligne['fichier']) && $ligne['fichier'] !== '' &&
                                   is_file(self::DOSSIER_LOGO_CLUB . $ligne['fichier']);
}
```

5.3. Le script index.html

L'interface statique se limite ici à une simple div identifiée 'lesCartes' sur laquelle les cartes générées dynamiquement viendront s'insérer.

```
<div id="lesCartes" class="liste-clubs"></div>
```

Cependant, la structure de chaque carte est définie dans une balise <template> qui sera utilisée comme modèle dans le script index.js pour générer rapidement chaque carte.

Les styles spécifiques utilisés dans le template sont définis dans la balise <style>

Les intérêts de passer par une balise <template> sont nombreux.

02 Développement Web> 03 Élément pratique > Utilisation de la balise HTML template

5.4. Le script index.php

Point d'entrée de la fonctionnalité, il doit récupérer les clubs et générer la page en instanciant un objet Page

5.5. Le script index.js

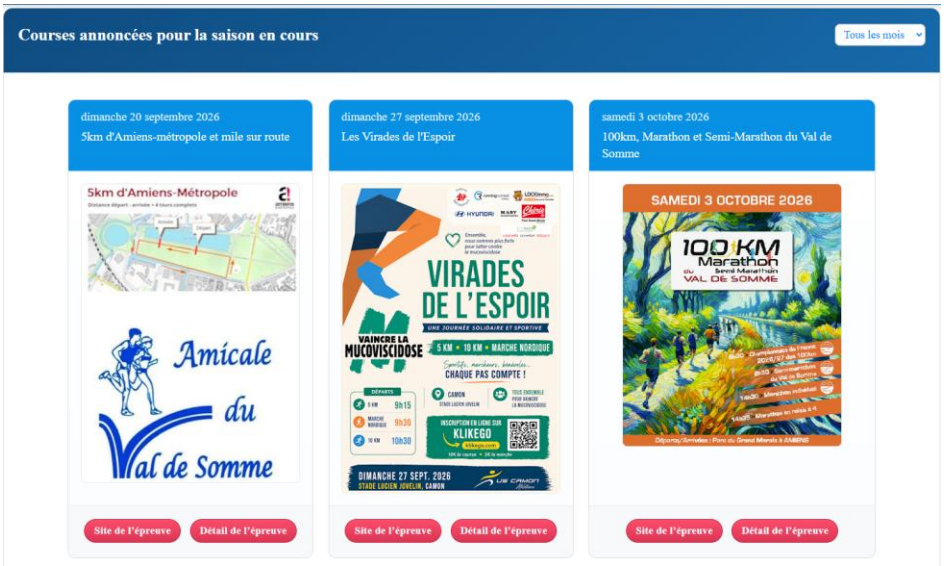
Il affiche les données sous la forme de carte en clonant le template défini dans le fichier index.html

6. Vérifier le fonctionnement de cette interface et son adaptation sur smartphone.

6. Les annonces

Répertoire : consultation/annonce/

6.1. L'interface



La zone de liste contient tous les mois, mais les mois sans annonce sont grisés.

6.2. La classe Annonce

Nous avons besoin de récupérer les annonces 'actives' dont la date n'est pas dépassée.

```
Select nom, description, date, date_format(date, '%d/%m/%Y') as dateFr, affiche, url
from annonce
 where date >= curdate()
 order by date;
```

De plus il faut comme pour les clubs, vérifier que l'affiche est bien présente dans le répertoire public/data/annonce

```
foreach ($lesLignes as &$ligne) {
      $ligne['present'] = isset($ligne['affiche']) && $ligne['affiche'] !== '' &&
                          is_file(self::DOSSIER_ANNONCE . $ligne['affiche']);
}
```

Point important : Le rôle de la couche métier devrait se limiter à récupérer des données brutes sans les modifier. La boucle foreach devrait se trouver dans une couche intermédiaire entre le contrôleur et la couche métier.
Il serait même souhaitable ici de laisser cette tache au client.

6.3. Le script index.html

Il comporte :

- un conteneur principal 'lesCartes' ;
- une balise <template> pour le modèle de carte ;
- une balise <style> pour les styles spécifiques utilisé dans la balise <template> ;
- une balise <select> statique contenant les mois de l'année afin de filtrer les annonces ;
- une fenêtre modale réalisée avec les classes Bootstrap afin d'afficher le détail de l'annonce.

6.4. Le script index.js

Le script gère l'affichage dynamique d'une liste d'annonces sous forme de cartes interactives, avec les fonctionnalités suivantes :

Récupération des données : Il utilise getData('lesAnnonces') pour obtenir la liste des annonces injectées par Vue.
Filtrage par mois : Un sélecteur (filtreMois) permet de filtrer les annonces en fonction du mois sélectionné.
Création de cartes : Pour chaque annonce, il clone un template HTML (modeleAnnonce), remplit les informations (date, nom, description, image, etc.), et ajoute des boutons pour accéder au site de l'épreuve ou afficher les détails dans une modale Bootstrap.
Affichage conditionnel : Si aucune annonce n'est disponible, il affiche un message personnalisé via genererMessage.}

7. Vérifier le fonctionnement de cette interface et son adaptation sur smartphone.

02 Développement Web> 03 Élément pratique > Contrôler l'affichage des images associées aux enregistrements d'une table

8. Réalisation les modifications nécessaires afin de transférer la vérification vers le client

7. Les projets

Répertoire : consultation/projet/



La particularité réside dans la source de données qui doit contenir les projets et leurs compétences.

La classe métier Projet regroupent ainsi les méthodes permettant de récupérer des informations à la fois la table 'projet' et la table 'competenceprojet'. Elle utilise aussi la table 'competence' afin d'obtenir le libellé des compétences.

Le contrôleur index.php récupère et transfère les données côté client en appelant la méthode Projet::getProjetEtCompetences() qui réalise la jointure entre ces trois tables

```
select projet.id, nom, competenceprojet.idCompetence,
        concat('C.', idBloc, '.', idDomaine, '.', competence.idCompetence) as code, libelle
from projet
      join competenceprojet on projet.id = competenceprojet.idProjet
      join competence on competence.id = competenceprojet.idCompetence
order by nom;
```

La mise en forme doit donc ici être adaptée. La présentation sous forme de tableau n'est pas adaptée ni même la présentation sous forme de carte au regard de la longueur des libellés des compétences.

Le choix se porte alors sur une représentation hiérarchique :

Niveau 1 : le nom du projet

Niveau 2 : Le code et le libellé des compétences sous la forme d'un tableau

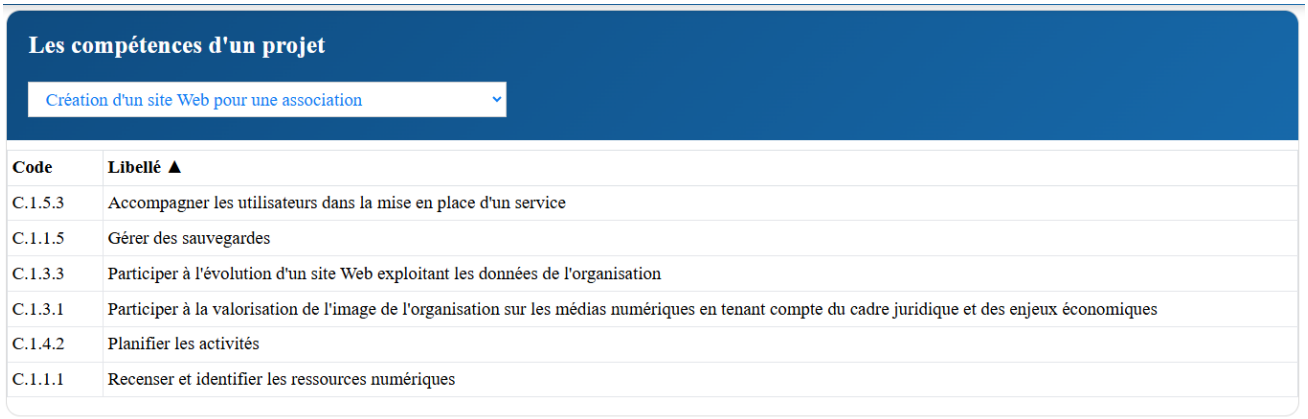
La mise en forme est donc réalisée par le script index.js puisqu'elle est entièrement dynamique

9. En analysant le script index.js, indiquer les instructions qui permettent d'afficher ou de masquer les compétences d'un projet.

8. Les compétences associées à un projet

Répertoire : consultation/competenceprojet/

8.1. L'interface



Le contrôleur index.php récupère et transfère les données côté client en appelant la méthode Projet::getListe()

Ces données sont utilisées côté client pour alimenter la zone de liste.

A chaque changement de valeur sélectionnée au niveau de la zone de liste, il faut interroger le serveur pour obtenir la liste des compétences associées au projet sélectionné.

Rappel : au niveau du fichier index.html, les colonnes du tableau des compétences utilisent les attributs "data" **data-champ** et **data_type** afin d'utiliser la fonction activerTri() du composant /composant/fonction.tableau.js qui permet de trier le tableau en cliquant sur l'entête de la colonne.

data-champ permet d'indiquer la propriété des éléments du tableau Javascript sur lequel le tri sera effectué.

data-type permet de définir le type des données à trier (text par défaut, date ou number)

8.2. Le script index.js

Il permet de **sélectionner un projet et d'afficher automatiquement les compétences qui lui sont associées**. Lorsqu'un projet est choisi, une requête AJAX récupère ses compétences et met à jour le tableau. Le tableau peut ensuite être **trié par colonne**, et un message s'affiche si aucun projet n'est enregistré

Lorsque l'utilisateur sélectionne un autre projet, il faut interroger le serveur (script ajax/getlescompetences.php) pour récupérer les compétences associées au projet sélectionné.

Nous utiliserons toujours la fonction appelAjax du composant fonction/ajax.js
Cette fonction encapsule l'API fetch de JavaScript, qui permet d'envoyer des requêtes HTTP depuis le navigateur.

Elle intègre l'envoi du jeton, et réalise un pré-traitement de la réponse en automatisant toute la gestion d'erreurs.

Seule la réponse en cas de succès doit être traité par le script index.js.

***02 Développement Web> 03 Élément pratique >* 05 Utilisation de la fonction appelAjax()**

Cette approche offre plusieurs avantages :

Expérience utilisateur fluide : Pas de rechargement de page, ce qui rend l'interface plus réactive.
Efficacité : Seules les données nécessaires sont échangées entre le client et le serveur.
Modularité : Le code JavaScript peut traiter dynamiquement les réponses du serveur.

8.3. Le contrôleur ajax/getlescompetences.php

Il se charge de vérifier la méthode d'appel (POST), le paramètre attendu (idProjet) et retourne dans le format JSON la réponse.

Il utilise pour cela deux méthodes de la classe Projet :

getLesCompetences($idProjet) pour retourner les compétences associées au projet dont l'id est passé en paramètre

getById($id) pour vérifier l'existence du projet

Pour garantir que ce contrôleur ne peut être appelé qu'en passant par le contrôleur principal, un jeton est mis en place.

***02 Développement Web> 03 Élément pratique >* 04 Mise en place d'une protection des scripts Ajax par un jeton d'accès**

***02 Développement Web> 03 Élément pratique >* 06 Les scripts PHP appelés en Ajax**

10. Vérifier le fonctionnement de ce script à l'aide du logiciel Insomnia

06 Outils utilisés> Insomnia